

NAME: SAHIL KOCHAREKAR

ENROLLMENT: MITU22BTCS0689

SUBJECT: HPCL

ASSIGNMENT 3

Question 1: Write a C program that adds two integer arrays A and B, each containing N elements, and stores the result in array C. Parallelize the addition using parallel for.

CODE:

```
#include <stdio.h>
#include <omp.h>

#define N 10

int main() {
    int A[N], B[N], C[N];

    for (int i = 0; i < N; i++) {
        A[i] = i + 1;
        B[i] = (i + 1) * 2;
    }

    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }

    printf("Array C after parallel addition:\n");
    for (int i = 0; i < N; i++) {
        printf("%d ", C[i]);
    }

    return 0;
}
```

Output:

```
Enter number of elements: 2
Enter elements of A:
23
45
Enter elements of B:
45
45
Result C:
68 90

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 2: Write a C program that calculates the sum and average of N floating-point numbers stored in an array. Identify the shared variable and parallelize it using reduction.

CODE:

```
#include <stdio.h>
#include <omp.h>

#define N 5

int main() {
    float A[N] = {10.5, 20.0, 30.5, 40.0, 50.0};
    float sum = 0.0, average;

    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < N; i++) {
        sum += A[i];
    }

    average = sum / N;

    printf("Sum = %.2f\n", sum);
    printf("Average = %.2f\n", average);

    return 0;
}
```

Output:

```
Sum = 151.00
Average = 30.20

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 3: Write a C program that finds the largest value in an integer array and reports its index.

CODE:

```
#include <stdio.h>
#include <limits.h>
#include <omp.h>

#define N 10

int main() {
    int A[N] = {12, 45, 7, 90, 23, 56, 34, 78, 11, 65};
    int max = INT_MIN;
    int maxIndex = -1;

    #pragma omp parallel
    {
        int localMax = INT_MIN;
        int localIndex = -1;

        #pragma omp for nowait
        for (int i = 0; i < N; i++) {
            if (A[i] > localMax) {
                localMax = A[i];
                localIndex = i;
            }
        }

        #pragma omp critical
        {
            if (localMax > max) {
                max = localMax;
                maxIndex = localIndex;
            }
        }
    }

    printf("Largest value = %d\n", max);
    printf("Index of largest value = %d\n", maxIndex);

    return 0;
}
```

OUTPUT:

```
Largest value = 90
Index of largest value = 3

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 4: Write a C program that multiplies two square matrices, A and B, of size $N \times N$ and stores the result in matrix C.

CODE:

```
#include <stdio.h>
#include <omp.h>

int main() {
    int N, i, j, k;

    printf("Enter matrix size N: ");
    scanf("%d", &N);

    int A[N][N], B[N][N], C[N][N];

    printf("Enter matrix A:\n");
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            scanf("%d", &A[i][j]);

    printf("Enter matrix B:\n");
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            scanf("%d", &B[i][j]);

    #pragma omp parallel for collapse(2)
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            C[i][j] = 0;

            for (k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    printf("Result matrix C:\n");

    for (i = 0; i < N; i++) {
```

```

        for (j = 0; j < N; j++)
            printf("%d ", C[i][j]);

        printf("\n");
    }

    return 0;
}

```

OUTPUT:

```

Enter matrix size N: 1
Enter matrix A:
1
Enter matrix B:
2
Result matrix C:
2

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 5: Write a C program that counts how many prime numbers exist between 2 and a user-provided upper limit N.

CODE:

```

#include <stdio.h>
#include <omp.h>

int main() {
    int N, i, j;
    int count = 0;

    printf("Enter upper limit N: ");
    scanf("%d", &N);

    #pragma omp parallel for reduction(+:count)
    for (i = 2; i <= N; i++) {

        int isPrime = 1;

        for (j = 2; j * j <= i; j++) {
            if (i % j == 0) {
                isPrime = 0;
                break;
            }
        }

        if (isPrime) count++;
    }
}

```

```
        if (isPrime)
            count++;
    }

    printf("Number of prime numbers between 2 and %d = %d\n",
        N, count);

    return 0;
}
```

OUTPUT:

```
Enter upper limit N: 100
Number of prime numbers between 2 and 100 = 27
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```